

CMSC 603

High-Performance Distributed Systems

Introduction



Dr. Preetam Ghosh
Professor

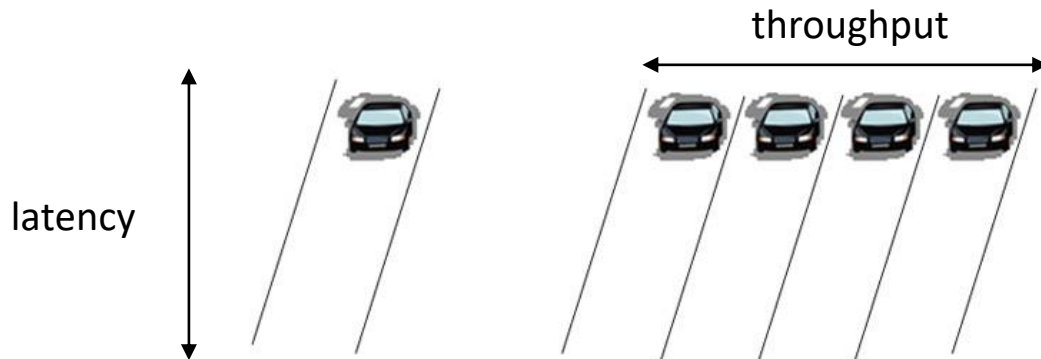
Department of Computer Science
pghosh@vcu.edu

Course main idea:

- Understand the **power** and **limitations** of parallel and distributed computing under temporal, spatial, and memory constraints for data-intensive applications on heterogeneous hardware infrastructures and software frameworks

How to make programs run faster:

- Decrease the latency (time to complete a task)
- Increase the throughput (number of tasks completed per time unit)



Decreasing the latency is usually difficult:

- Limited by the nature of the code and the hardware architecture
- **Solution 1:** Profile and rewrite the code
 - Algorithm computational complexity and memory access patterns
 - Cost and reward trade-off
- **Solution 2:** Improve the hardware
 - Increase the frequency of the processor
 - Specialized hardware: FPGAs, GPUs, Tensor cores, Ray tracing
- CPUs have been evolving towards more cores rather than raw speed
 - Desktop: 8 or 16 cores in a CPU but frequency ~ 4 GHz

Increasing the throughput. Barriers to parallelization:

- Embarrassingly parallel problems are trivial
- Some sequential problems may be parallelizable by rewriting code, others no
- Data/task dependencies. Unable to start a task until the previous task finishes
- Synchronization and combination of distributed results
- Difficulty to debug since execution may happen in any order
- Security: Spectre/Meltdown vulnerabilities exploiting speculative execution
- Sequential tasks limit the maximum speedup. No matter how many processors, you won't be able to speed up any further (Amdahl's Law)

Limitations of speedup:

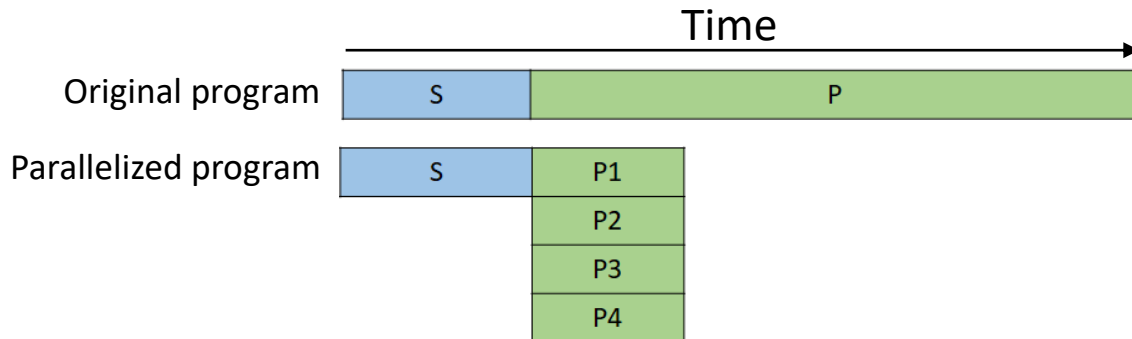
- All programs have sequential parts and parallel parts

S: proportion of serial runtime in an execution

P: proportion of parallel runtime in an execution

$$(S + P = 1)$$

- What's the speedup of a program using 1, 2, 4, ..., N processors?



Amdahl's Law (1967):

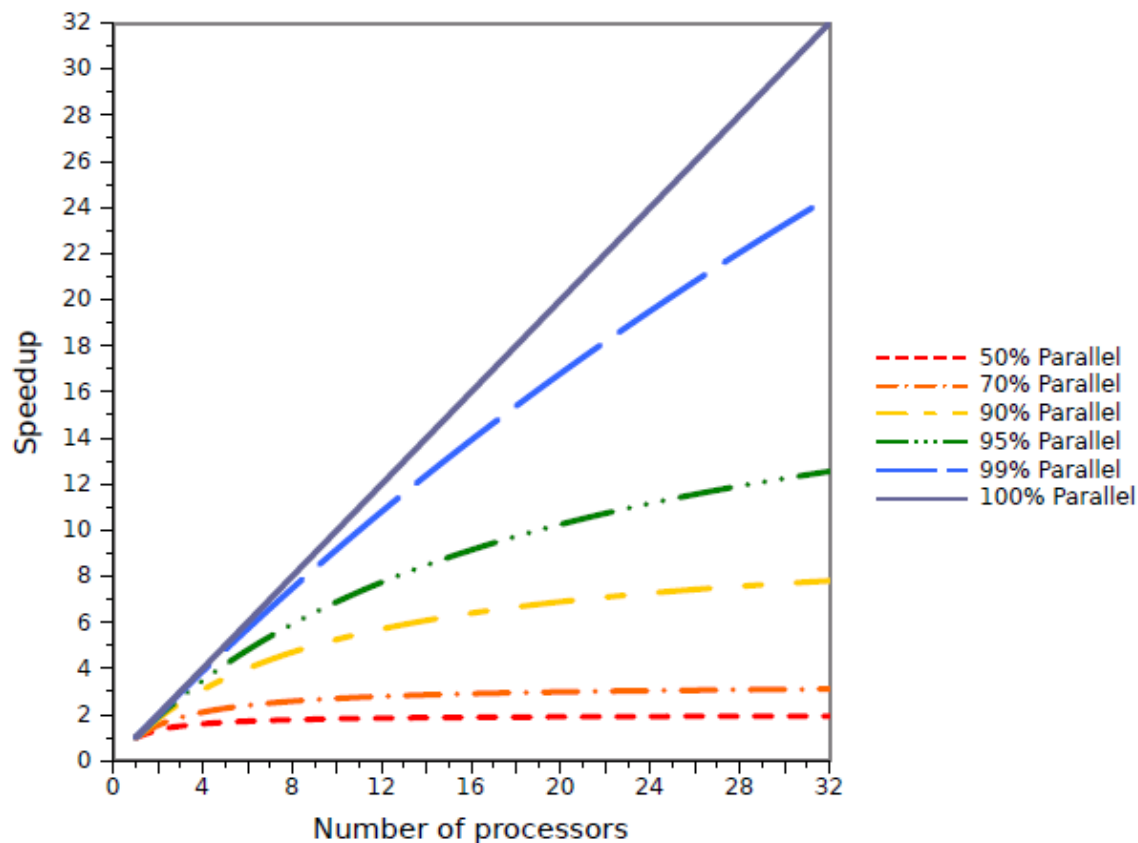
- Theoretical speedup of the execution of a task at **fixed** workload

$$Speedup = \frac{1}{S + \frac{P}{N}}$$

Maximum asymptotic speedup = $\frac{1}{S}$ since $\frac{P}{N} \rightarrow 0$

S	P	N	Speedup
1	0	1	1
0.5	0.5	8	1.77
0.1	0.9	10	5.26
0.01	0.99	256	72.11
0.001	0.999	∞	1,000

Fixed-size problem scaling, varying the fraction of parallel code:



Calculating the runtime and speedup:

T_S total time for the program to run in **serial** mode

T_P total time for the program to run in **parallel** mode

N number of processors or parallel execution units

- Under perfect conditions (assuming no **overhead** for parallelizing):

$$Speedup = \frac{T_S}{T_P} = \frac{1}{S + \frac{P}{N}}$$

- Real-world: no free lunch, overheads, dependencies, synchronization

Amdahl's Law generalization:

- The program may have many sequential and parallel sections
- Each of which we can tune to a different degree

$f_1, f_2, \dots, f_n$: fraction of time in section n

$Sf_1, Sf_2, \dots, Sf_n$: speedup for section n

$$Speedup = \frac{1}{\frac{f_1}{Sf_1} + \frac{f_2}{Sf_2} + \dots + \frac{f_n}{Sf_n}}$$

- Consider a program with 4 sections. Which option is better?

Section	Fraction of Runtime	Speedup Option 1	Speedup Option 2
1	0.55	1	2
2	0.25	5	1
3	0.15	3	1
4	0.05	10	1

Amdahl's Law generalization:

- Option 1
$$speedup = \frac{1}{0.55 + \frac{0.25}{5} + \frac{0.15}{3} + \frac{0.05}{10}} = 1.53$$

- Option 2
$$speedup = \frac{1}{\frac{0.55}{2} + 0.45} = 1.38$$

- Performance gain vs effort-cost **trade-off**
 - Coding time and human resources required to achieve a small speedup
 - Code optimization & parallelization takes time and money to implement
- Overhead consideration, especially **network latency** in distributed systems

Empirically estimating the proportion of parallel code and maximum parallel speedup:

- Quick way to estimate the proportion of parallel code (P-estimated)
- Run the program forcing 1 core and N cores, and compute the relative speedup

$$Speedup = \frac{T_S}{T_P} \quad P\text{-estimated} = \frac{\frac{1}{Speedup} - 1}{\frac{1}{N} - 1}$$

Example: T_S 500 s, T_P 50 s, N 20
Speedup = 10
P-estimated = 0.94736

- Estimating the speedup using another M cores = $\frac{1}{\left((1 - P_{estimated}) + \frac{P_{estimated}}{M}\right)}$

Example: M 200
Speedup = 17.42
 TP = 28.68 s

Gustafson's Law:

- Speedup predictions according to Amdahl's law are pessimistic
- Amdahl's law assumed the fraction of parallelizable code is **fixed**
- Gustafson: **parallelism weight increases when the problem size (n) increases**

$S(n)$: serial runtime portion for a problem size n

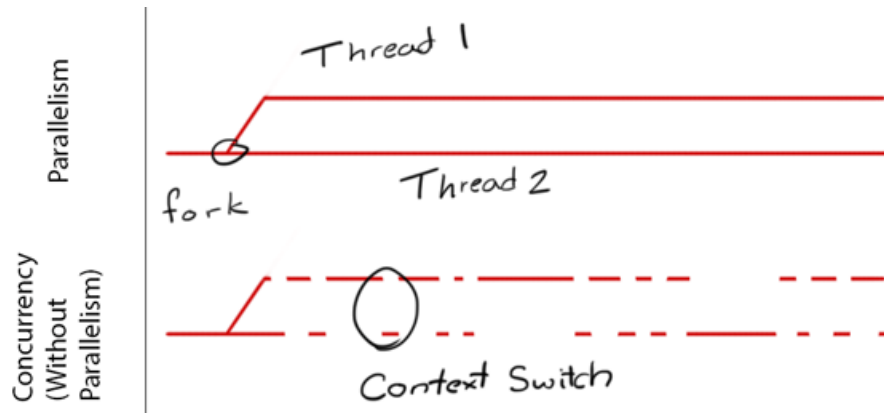
$P(n)$: parallel runtime portion for a problem size n

$$\text{Total runtime} = S(n) + (P(n) / N)$$

- Different sections of the code scale differently to the increase of data size
- Serial and overhead times become irrelevant for very large complex problems

Concurrent vs parallel vs distributed computing:

- **Concurrency** is when two or more tasks can start, run, and complete in overlapping time periods. It doesn't necessarily imply they'll ever be running simultaneously at the same point in time. Does not require multiple processors. Keyword: *Interruptible*.
- **Parallelism** is when two or more tasks run simultaneously at the same point in time. Requires multiple processors. Keyword: *Independency*.
- **Distributed** is when tasks execute on autonomous and interconnected computing systems. Keyword: *Interconnection and isolation*.

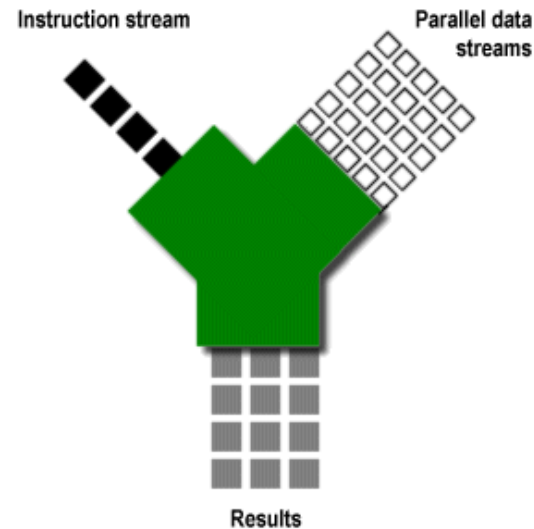


Levels of parallelism:

- Data-level vs instruction-level parallelism
- Thread-level vs process-level parallelism

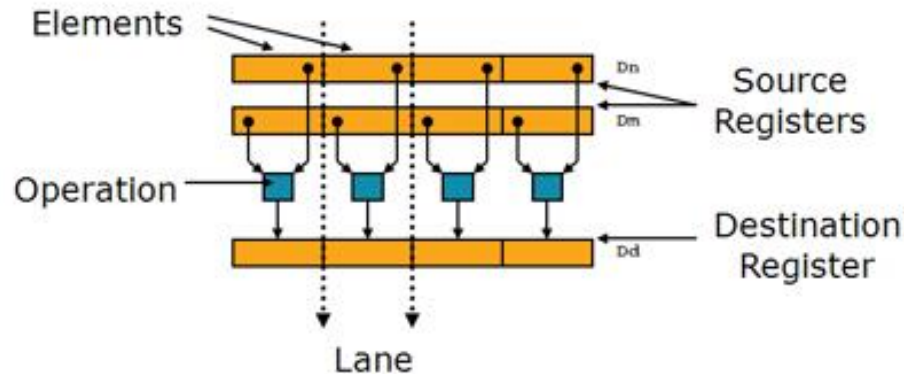
Flynn's taxonomy:

- SISD (Single Instruction Single Data)
- SIMD (Single Instruction Multiple Data)
- MIMD (Multiple Instruction Multiple Data)



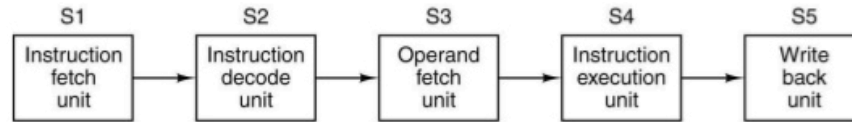
Data-level parallelism:

- Perform identical operations on **lots of independent** data
- Vector processor arrays, FPGAs, GPUs, Streaming SIMD Extensions (SSE)
- Drawbacks: portability and compatibility

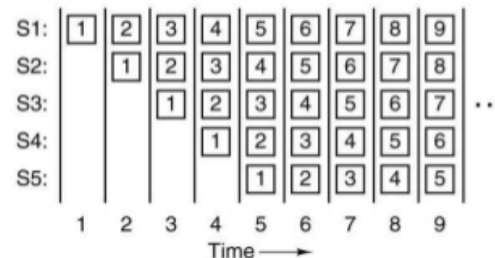


Instruction-level parallelism:

- Exploiting overlap among instructions
- Pipelining, e.g. simultaneous ALU execution / data transfer
- Superscalar CPUs

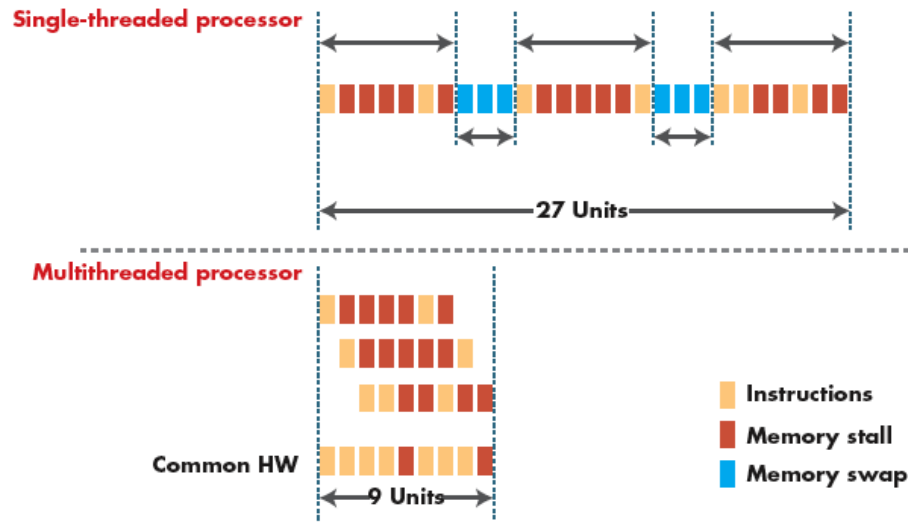


(a)



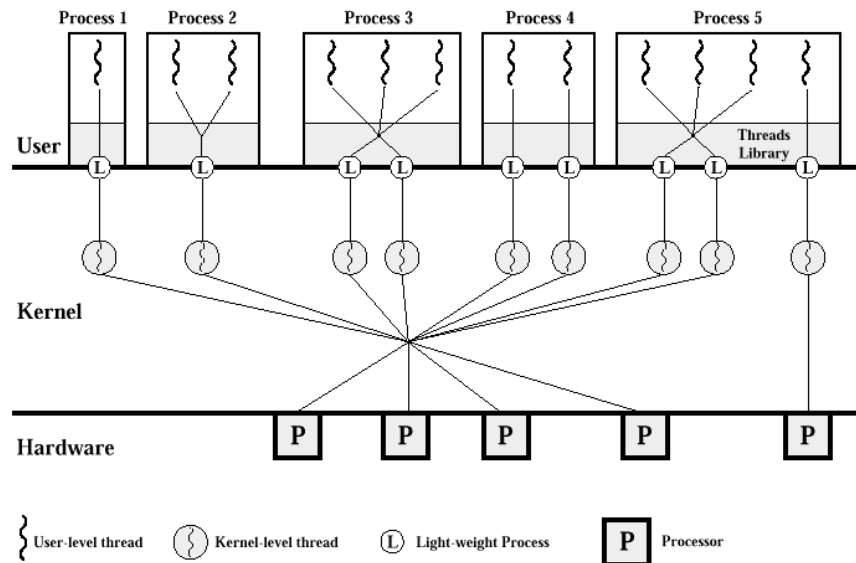
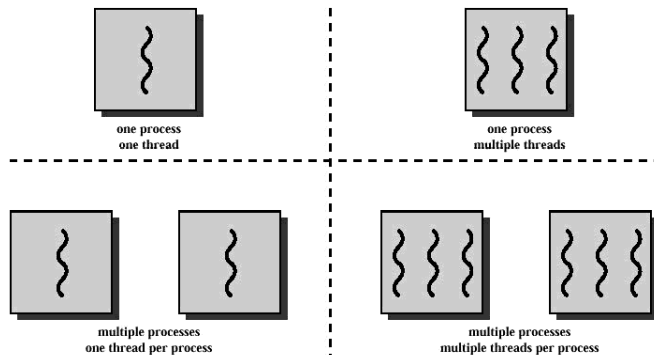
Thread-level parallelism:

- A process may contain multiple threads
- Threads **share the memory space of the process**
- Mutual exclusion mechanisms to control synchronization among threads and critical sections
- The operating system schedules threads and processes



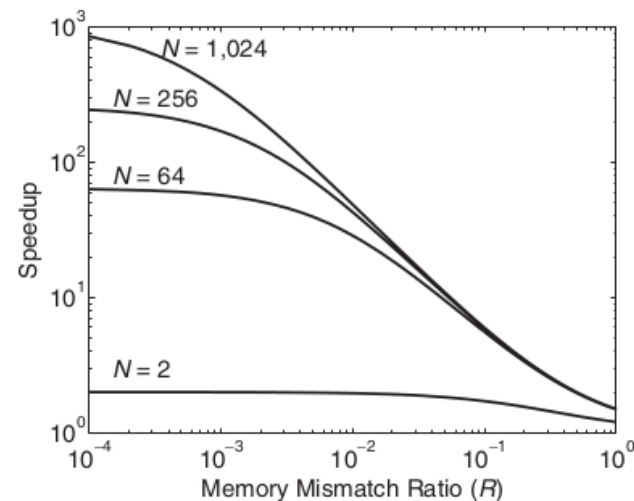
Process-level parallelism:

- Multiple independent and cooperating processes in a system
- Processes have **separate** memory spaces
- Message communication (MPI)



Overheads:

- Creating / destroying threads vs processes
- Network delay is orders of magnitude slower than intra node communication
- Memory bandwidth, latency, and memory access pattern
- Memory collisions and RAW/WAW conflicts
- Memory hierarchy
 - Register $\leftrightarrow$ cache $\leftrightarrow$ RAM $\leftrightarrow$ disk
 - Memory-bound programs are limited by bandwidth and cache misses



Enhancing performance (hardware and compiler):

- Cache memory is useful if the code shows temporal and spatial locality
- Data load/store transactions take place using full blocks of data (bus width)
- Pipelining and segmentation
- Asynchronous execution & data transfer
- Branch prediction & speculation, out of order execution
- Loop unrolling
- Memory layout of multi-dimensional arrays

Enhancing performance (software):

- Understand an algorithm's computational complexity
- Optimize memory access patterns, data types, and structures
- Optimize arrays size and alignment (padding)
- Minimize I/O interruptions (no prints while calculating)
- Optimize conditional statements (help the compiler)
- Use functions calls properly (inline)
- Binary search or sequential search, sorting, min/max, (problem size dependent)
- Profile your code to identify performance bottlenecks

Distributed systems

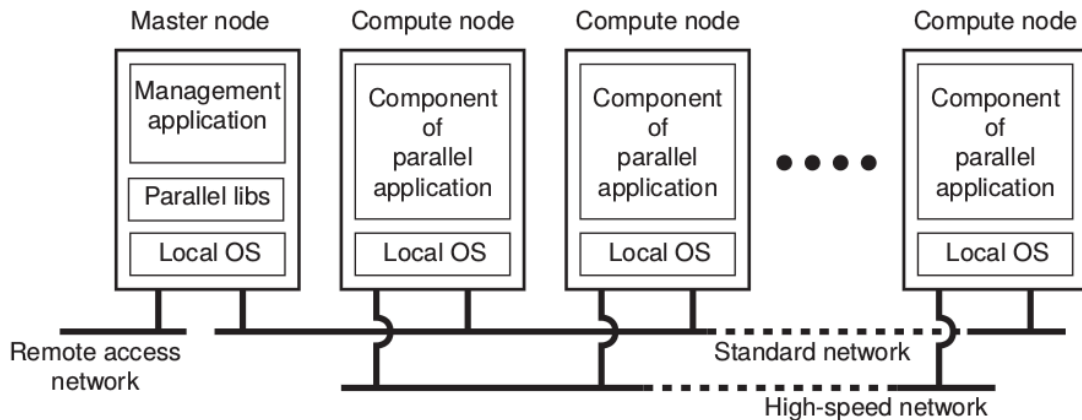
- A distributed system is a collection of autonomous computing elements that appear to the users as a single coherent system
- Transparency: access, location, migration, concurrency, failure
- Flexibility: easier to change, update, and upgrade
- Reliability: availability, maintenance, fault tolerance
- Performance: communication delays, transparency cost
- Scalability: plug & play computing elements, dynamic problem size
- Security: whole course on security of distributed systems

Scaling distributed systems

- **Vertical** (power of nodes) and **horizontal** (number of nodes) **scalability**
- Hide communication latencies. Asynchronous communication
- Distribution. Partition data and computation across machines
- Replication. Mirroring servers. Issues with inconsistencies and synchronization
- False assumptions:
 - Network is reliable, secure, homogeneous
 - Latency is zero, bandwidth is infinite
 - Migration cost is zero

Distributed computing systems

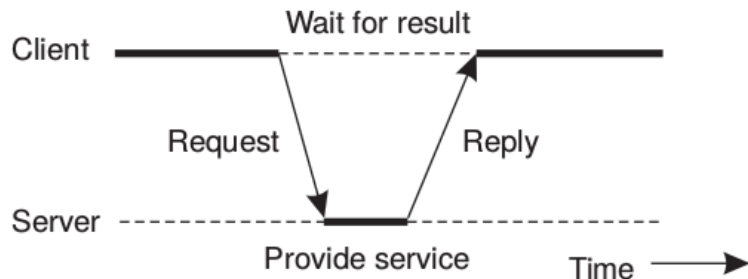
- Group of homogeneous high-end systems connected through high-speed Ethernet or InfiniBand. One master node and multiple compute nodes
- Job submission system, e.g. **slurm**



- Grid and cloud computing: dispersed across multiple locations

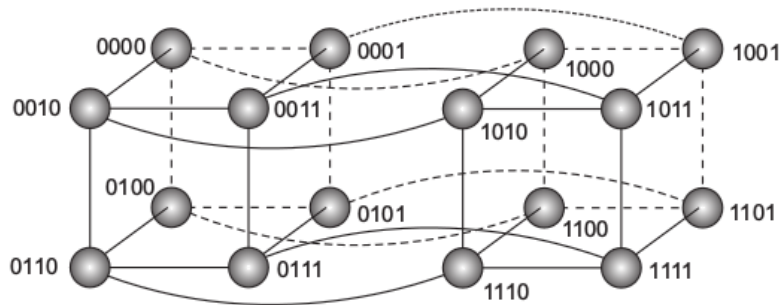
Centralized architectures: client-server model

- There are processes offering services (high-end servers), producers
- There are processes that use services (thin clients), consumers
- Clients and servers can be on the same/different machines
- Clients follow request/reply model for using services
- Servers usually delegate service into other servers



Decentralized architectures: peer-to-peer systems

- Structured / unstructured P2P topology: federation
- Organize the nodes in a structured overlay network such as a ring or hypercube, and make specific nodes responsible for services based only on their ID
- Requires efficient P2P routing. Flooding, random walk. Superpeer



- Trilemma: Decentralization, Security, Scalability

CMSC 603

High-Performance Distributed Systems

Introduction



Dr. Preetam Ghosh
Professor

Department of Computer Science
pghosh@vcu.edu